Ders 4 Çalışma Özeti

Ders 4 Çalışma Özeti

Genel Konular

- Bilgi teorisi: olasılık ve bilgi (ekrandan, 00:30:00)
  - Bir olayın gerçekleşme ihtimali ne kadar düşükse taşıdığı bilgi o kadar fazladır; ihtimali yüksek olan olay az bilgi taşır. Slayttaki örnek: sıfır derecede kar yağması çok olasıdır ve fazla bilgi taşımaz; 20 derecede kar yağarsa olağanüstü bir durumdur ve çok bilgi taşır.
  - Sıkıştırmadaki karşılığı: kaynakta sık geçen sembol daha az bitle, nadir geçen sembol daha fazla bitle kodlanır. Hoca bunu "bu dersin bütün amacı" diye özetledi.
  - Sezgisel örnek: 100 tane `a` ve 2 tane `j` içeren bir dokümanda `a` 1 bitle, `j` 10 bitle kodlanırsa toplam $100 \cdot 1 + 2 \cdot 10 = 120$ bit harcanır. ASCII ile $102 \cdot 8 = 816$ bit gerekirdi.
- Bir olayın bilgisi (self-information) (ekrandan, 00:30:00 ve 00:40:00)
  - $I(A) = \log_x \dfrac{1}{P(A)} = -\log_x P(A)$. $P(A)$: A olayının gerçekleşme ihtimali, $I(A)$: A olayının taşıdığı bilgi.
  - Bilgi ile olasılık ters orantılı olduğu için $1/P(A)$ alınır. Logaritma değeri küçültüp ölçülebilir hâle getirir. Taban 2'dir, çünkü dünya dijitaldir (0 ve 1) ve birim bittir.
  - $\log_2 P(A)$ negatif çıkacağı için önündeki eksi bilgiyi pozitif yapar.
- Para atma örnekleri (ekrandan, 00:40:00)
  - Hilesiz para: $P(Y) = P(T) = 1/2$, $I(Y) = I(T) = \log_2 2 = 1$ bit.
  - Sahte para: $P(Y) = 1/8$, $P(T) = 7/8$. $I(Y) = \log_2 8 = 3$ bit, $I(T) = \log_2(8/7) \approx 0{,}193$ bit.
  - Pratikte 0,193 bit diye bir şey yoktur; tura en az 1 bitle kodlanır. Teorideki değere pratikte ulaşılamaması sıkıştırmanın hiçbir zaman ideal olamamasının sebeplerinden biridir.
- Entropi (ekrandan, 00:40:00 ve 00:44:10)
  - Bağımsız olaylardan oluşan set $S = \bigcup_i A_i$ için ortalama bilgi entropidir: $H = \sum_i P(A_i)\, I(A_i) = -\sum_i P(A_i) \log_2 P(A_i)$.
  - Entropi, olaylar kodlandığında olması gereken ideal ortalama binary sembol uzunluğudur: bir sembolü ortalama kaç bitle kodlayacağımızı söyler. Örneğin $H = 4{,}5$ ise bazı semboller 1, bazıları 7 bitle kodlanır ama ortalama 4,5 bit olur.
  - Shannon'a göre en iyi kayıpsız sıkıştırmada kodlanan sembollerin ortalama uzunluğu kaynağın entropisine eşit olmalıdır; ancak bu ideal durum pratikte mümkün değildir.
- Entropi örneği: dört sembollü kaynak (ekrandan, 00:44:10)
  - $P(a_1) = 0{,}49$, $P(a_2) = 0{,}25$, $P(a_3) = 0{,}25$, $P(a_4) = 0{,}01$.
  - $H = -(0{,}49 \log_2 0{,}49 + 0{,}25 \log_2 0{,}25 + 0{,}25 \log_2 0{,}25 + 0{,}01 \log_2 0{,}01) = 1{,}57$ bit.
  - Her sembol 2 bitle (`00`, `01`, `10`, `11`) kodlanırsa ortalama mesaj uzunluğu $S = 2$ bit olur.
- Ortalama mesaj uzunluğu ve artıklık (redundancy)
  - S: pratikte bulunan kodların ortalama uzunluğu. Artıklık $|S - H|$'dir; örnekte $|2 - 1{,}57| = 0{,}43$.
  - Artıklık sıfıra ne kadar yakınsa yöntem o kadar iyidir; verinin içinde sıkıştırılacak bir şey kalmamıştır. Büyükse ya yöntem değiştirilir ya da arkasından ikinci bir sıkıştırma uygulanır.
- 2 tabanında logaritma (ekrandan, 00:50:00)

  - $\log_2 x = q \Rightarrow x = 2^q \Rightarrow \ln x = q \ln 2 \Rightarrow q = \dfrac{\ln x}{\ln 2}$. Hesap makinesinde $\log_2 x = \dfrac{\log_{10} x}{\log_{10} 2}$ ile hesaplanır.
- Örnek: 16 sayılık dizi ve rezidü (residual) dizisi (ekrandan, 00:54:35 ve 01:04:45)
  - Dizi: `1 2 3 2 3 4 5 4 5 6 7 8 9 8 9 10`. 1, 6, 7, 10 birer kez ($1/16$), 2, 3, 4, 5, 8, 9 ikişer kez ($2/16 = 1/8$) geçer. $H = 4 \cdot \frac{1}{16} \cdot 4 + 6 \cdot \frac{1}{8} \cdot 3 = 3{,}25$ bit.
  - Her eleman bir öncekinden çıkarılırsa (ilk eleman olduğu gibi kalır) rezidü dizisi elde edilir: `1 1 1 -1 1 1 1 -1 1 1 1 1 1 -1 1 1`. $P(1) = 13/16$, $P(-1) = 3/16$, $H = 0{,}70$ bit.
  - Alıcı orijinale $x_n = x_{n-1} + r_n$ ile döner (x_n: orijinal dizinin n. elemanı, r_n: rezidü dizisinin n. elemanı). Örnek: $x_2 = 1 + 1 = 2$, $x_3 = 2 + 1 = 3$, $x_4 = 3 + (-1) = 2$.
  - Alıcı, gelen verinin orijinal değil rezidü olduğunu bilmelidir; bu bilgi ve açma kuralı başlığa (header) yazılır ya da iki taraf önceden anlaşır.
- Statik ve dinamik model (ekrandan, 01:04:45)
  - $x_n = x_{n-1} + r_n$ statik modeldir, çünkü parametreler n değiştikçe değişmez. Verinin karakteristiğine bağlı olarak parametreler n ile değişiyorsa (örneğin n tekken bir işlem, çiftken başka bir işlem) dinamik modeldir.
- Örnek: 20 sayılık dizi ve bloklama (ekrandan, 01:10:00 ve 01:15:40)
  - Dizi: `1 2 1 2 3 3 3 3 1 2 3 3 3 3 1 2 3 3 1 2`. RLE ile 26 sayı çıkar (`1 1 1 2 1 1 1 2 4 3 ...`); girdi 20 sayı olduğundan bu veriye RLE uymaz.
  - Tek tek sembol: $P(1) = P(2) = 1/4$, $P(3) = 1/2$, $H = 1{,}5$ bit/sembol; 20 sembol için 30 bit. 2 bitlik sabit kodla $S = 2$, artıklık $0{,}5$.
  - İkişerli bloklar (`12` ve `33`): 10 blok, $H = 1$ bit/blok; 10 bit yeter. `12` için 0, `33` için 1 verilirse $S = 1$ ve artıklık 0 olur.
- Veri modelleri (ekrandan, 01:40:00)
  - Fiziksel model, olasılık modeli ve Markov model. Verinin modelini doğru kurmak kaynağın entropisini tahmin etmek ve etkili bir sıkıştırma algoritması seçmek için önemlidir.
  - Fiziksel modelde verinin nasıl üretildiği bilinir (ses kaydının sessiz odada mı kalabalıkta mı alındığı, bir hanenin elektrik tüketiminde evde kaç kişi yaşadığı gibi). Derste asıl olasılık ve Markov modelleri kullanılacaktır.
- Olasılık modeli (ekrandan, 01:43:40)
  - Ignorance model: kaynak hakkında hiçbir bilgi yoktur; her sembol bağımsız ve eşit olasılıklıdır (İngilizce için her harf $1/26$, Türkçe için $1/29$).
  - Probability model: her sembolün olasılığı geçme sıklığından hesaplanır; semboller yine bağımsızdır ama olasılıkları farklıdır. Alfabe $\{a_1, \ldots, a_m\}$, olasılıklar $\{P(a_1), \ldots, P(a_m)\}$. İstatistiksel sıkıştırma yöntemleri bu modeli kullanır.
- Markov model (ekrandan, 01:50:00)
  - Sembollerin olasılıkları birbirine bağımlı hesaplanır (adını Rus matematikçi A. A. Markov'dan alır). Kayıpsız sıkıştırmada kullanılır; "Discrete Time Markov Chain" olarak da bilinir.
  - k'ıncı dereceden Markov model: $P(x_n \mid x_{n-1}, \ldots, x_{n-k}) \approx P(x_n \mid x_{n-1}, \ldots, x_{n-k}, \ldots)$. Geçmişteki k sembolün bilgisi, geçmişin tüm bilgisine yaklaşık olarak eşittir.
  - Geçmişin tamamına bakmak hem hesap olarak imkansızdır hem de uzun geçmişlerin olasılığı çok küçüktür; pratikte k 2 ya da 3 alınır.
- Siyah-beyaz görüntü için Markov model (ekrandan, 02:00:00 ve 02:10:00)
  - İki durum (state) vardır: S_w (beyaz) ve S_b (siyah). Geçişler: $P(w|w)$, $P(b|w)$, $P(w|b)$, $P(b|b)$.
  - Markov modelin entropisi: $H = \sum_{i=1}^{M} P(S_i)\, H(S_i)$. Her durumun entropisi olasılık modeliyle hesaplanır.
  - Örnek: $P(S_w) = 0{,}8$, $P(S_b) = 0{,}2$, $P(w|b) = 0{,}3$, $P(b|w) = 0{,}01$.
    - Olasılık modeli: $H = -(0{,}8 \log_2 0{,}8 + 0{,}2 \log_2 0{,}2) = 0{,}722$ bit.
    - Markov: $H(S_b) = -(0{,}3 \log_2 0{,}3 + 0{,}7 \log_2 0{,}7) = 0{,}881$, $H(S_w) = -(0{,}01 \log_2 0{,}01 + 0{,}99 \log_2 0{,}99) = 0{,}081$, $H = 0{,}2 \cdot 0{,}881 + 0{,}8 \cdot 0{,}081 = 0{,}241$ bit.
  - Markov model daha düşük entropi verir, ama bu iki durumlu örnekte pratikte iki model de piksel başına 1 bit kullanır.
- Move-to-Front (MTF) kodlama (ekrandan, 02:20:00, 02:22:00 ve 02:24:00)

  - Sıkıştırma yönteminden önce uygulanan, lokal adaptif bir ön işlemdir. Sembolün alfabedeki sırası (indeks 0'dan başlar) yazılır, sonra sembol alfabenin önüne taşınır ve alfabe kaydırılır.
  - Girdi `a b c d d c b a m n o p p o n m`, alfabe `{abcdmnop}`. Kod: `0 1 2 3 0 1 2 3 4 5 6 7 0 1 2 3`, alfabenin son hâli `mnopabcd`. Entropi 2,81 bit/karakter; 0-7 arası değerler 3 bitle yazılırsa artıklık yaklaşık 0,19 bittir.
  - Kodlar Huffman ya da aritmetik kodlama gibi bir yöntemle sıkıştırılır. Slayttaki kod çözmede alfabenin son hâlinden başlanır ve girdi tersten elde edilir.
  - MTF yapılmadan yalnız alfabedeki sıra yazılırsa kod `0 1 2 3 3 2 1 0 4 5 6 7 7 6 5 4` olur; her değer iki kez geçtiği için entropi 3 bittir. MTF entropiyi düşürür.
- Kelimelerle MTF (ekrandan, 02:32:30)
  - Metin `the boy on my right is the right boy`. Yeni gelen kelime listenin sonuna eklenir, indeksiyle birlikte yazılır, sonra öne taşınır. Liste içinde olan kelime için yalnız indeks yazılır.
  - Çıktı: `0the 1boy 2on 3my 4right 5is 5 2 5`. Kelimeler bir kez göründükten sonra yalnız indeksleri yazıldığı için sıkıştırma sağlanır.

Hocanın Özellikle Vurguladığı Kısımlar

- Entropi, ortalama mesaj uzunluğu ve artıklık
  - Hoca bu dersin sonunda bu üç kavramın bilinmesi gerektiğini, her şeyin bunların üzerine kurulduğunu, geri kalan yöntemlerin "teferruat" olduğunu söyledi. Amaç her zaman artıklığı en aza indirmektir.
- Entropiyi unutmayın
  - Hoca "bu entropiyi kesinlikle unutmayın" dedi. Entropi teoriktir; pratikte kesirli bit olmadığı için ulaşılamaz.
- Veriyi tanımak
  - Uygun yöntemi seçmek için verinin içeriği tanınmalıdır. Fark (rezidü) almak ya da sembolleri bloklamak entropiyi ciddi biçimde düşürebilir. Yöntem veriyi genişletiyorsa (RLE'nin 20 sayıyı 26'ya çıkarması gibi) yanlış yoldayız demektir.
- Alıcıya bilgi vermek
  - Veri bir ön işlemden geçirildiyse (rezidü gibi) bu bilgi ve geri dönüş kuralı başlıkta alıcıya iletilmelidir; aksi hâlde alıcı açtığı veriyi orijinal sanar.
- Markov model ve olasılık modeli
  - Markov model her zaman daha iyi sonuç verir ama daha karmaşıktır ve kodlaması zordur. Bazı verilerde iki model arasında abartılacak bir fark yoktur.
- Prefix kod
  - Sonraki derste görülecek yöntemlerin hepsi prefix koddur. Bir kod prefix değilse sıkıştırılabilir ama açılamaz.

Kısa Tekrar Notları

- $I(A) = -\log_2 P(A)$; az olası olay çok bilgi taşır.
- $H = -\sum P(A_i) \log_2 P(A_i)$: ideal ortalama bit sayısı.
- S: pratikteki ortalama kod uzunluğu; artıklık $|S - H|$, sıfıra yakın olmalı.
- Hilesiz para: 1 bit; sahte para ($1/8$, $7/8$): 3 bit ve 0,193 bit.
- $\{0{,}49; 0{,}25; 0{,}25; 0{,}01\}$: $H = 1{,}57$, $S = 2$, artıklık $0{,}43$.
- $\log_2 x = \log_{10} x / \log_{10} 2$.
- `1 2 3 2 3 4 5 4 5 6 7 8 9 8 9 10`: $H = 3{,}25$; rezidü: $H = 0{,}70$; $x_n = x_{n-1} + r_n$.
- Statik model: parametreler n ile değişmez; dinamik model: değişir.
- `1 2 1 2 3 3 3 3 ...`: tek tek $H = 1{,}5$ (30 bit), 12/33 blokları $H = 1$ (10 bit).
- Modeller: fiziksel, olasılık (ignorance ve probability), Markov.
- Markov: k. derece, pratikte $k = 2$ ya da 3; $H = \sum P(S_i) H(S_i)$.
- Siyah-beyaz örnek: olasılık modeli 0,722 bit, Markov 0,241 bit.
- MTF: indeks yaz, sembolü öne taşı; örnekte $H = 2{,}81$, MTF'siz $H = 3$.
- Kelime MTF: `0the 1boy 2on 3my 4right 5is 5 2 5`.

Detaylı Açıklamalar

- **Neden logaritma ve neden eksi.** Bilgi doğrudan ölçülebilir bir şey değildir; olasılık ise sayılarak bulunur (adet bölü toplam). Bilgi olasılıkla ters orantılı olduğu için $1/P(A)$ alınır, logaritma ile ölçülebilir bir büyüklüğe çevrilir. Pay ve payda ile çalışmak zor olduğu için $\log(1/P) = -\log P$ yazılır. Olasılık 1'den küçük olduğu için $\log_2 P$ negatiftir; baştaki eksi bilgiyi pozitif yapar.
  - Ders sonrası not: Hoca 00:37:43'te slayttaki $H = \sum P(A_i)I(A_i)$ formülünün başına eksi koymayı unuttuğunu söyleyip eksi ekledi, 00:43:17'de ise eksinin I ile yazılan formülde değil, $\log_2 P(A_i)$ ile yazılan formülde olması gerektiğini söyledi. Doğrusu $H = \sum P(A_i)I(A_i) = -\sum P(A_i)\log_2 P(A_i)$'dir. 00:54:35 karesindeki $H = \sum P_i \log_2 P_i$ yazımında da eksi eksiktir.
- **Dört sembollü örnekte kod.** Hoca 2 bitlik kodları sayarken altyazıda semboller karışık geçiyor (00:44:46); slayta göre dört sembolün her birine 2 bitlik bir kod verilir ve $S = 2$ olur. Hoca bu sıkıştırma yöntemlerinin sonraki derslerde gösterileceğini, şimdilik yalnız sabit uzunluklu kod üzerinden artıklığın hesaplandığını söyledi.
  - Ders sonrası not: Hoca 00:42:29'da dördüncü sembolün olasılığını "0.1" diye okudu; slayttaki ve hesaptaki değer 0,01'dir.
- **16 sayılık dizi.** Slaytta $P(1) = P(6) = P(7) = P(10) = 1/6$ yazıyor (ekrandan, 00:54:35); doğrusu $1/16$'dır (16 sembolün her biri bir kez). $H = -[4 \cdot \frac{1}{16}\log_2\frac{1}{16} + 6 \cdot \frac{1}{8}\log_2\frac{1}{8}] = 1 + 2{,}25 = 3{,}25$ bit (python ile doğrulandı). Rezidü dizisinde yalnız 1 ve -1 kalır; $-(\frac{13}{16}\log_2\frac{13}{16} + \frac{3}{16}\log_2\frac{3}{16}) \approx 0{,}696$ bit. Pratikte iki sembol 1 bitle kodlanır; hoca $1 - 0{,}70 = 0{,}3$ artıklığın sıfıra yakın olduğunu söyledi.
- **20 sayılık dizi ve öğrenci önerisi.** Bir öğrenci tekrar sayısını yazmayı (RLE) önerdi. Hoca dizinin RLE ile 26 sayıya çıktığını, bayrak baytlı RLE versiyonunun da diziyi uzatacağını gösterdi. Sonra diziyi ikişerli bloklara ayırdı: `12` ve `33` dışında blok çıkmaz. Üçlü bloklarda (`121`, `233`, `331`, …) ve dörtlü bloklarda (`1212`, `3333`, `1233`, `3312`) daha çok farklı sembol çıkar; olay sayısı ne kadar azsa o kadar az bit gerekir.
  - Ders sonrası not: Hoca 01:13:01'de "1'den 4 tane, 2'den 4 tane" dedi; dizide 1 ve 2 beşer kez, 3 on kez geçer ($5/20 = 1/4$, $10/20 = 1/2$; python ile doğrulandı). Blok örneğinde slayt $P(12) = P(33) = 1/4$ yazıyor ve hoca $5/20$ dedi; 10 bloğun 5'i `12`, 5'i `33` olduğundan doğrusu $P(12) = P(33) = 5/10 = 1/2$'dir. Hoca karede $5/10$'u işaretledi (ekrandan, 01:15:40). $H = 1$ bit/blok sonucu doğrudur.
  - Ders sonrası not: 01:14:15'te altyazıda 2 ve 3'ün ikisi de `01` ile kodlanmış geçiyor; 2 bitlik kodda üç sembol farklı kodlar almalıdır (örneğin `00`, `01`, `10`).
- **Markov modelin derecesi.** Hoca iki durumlu siyah-beyaz modeli "ikinci dereceden" diye andı ve üçüncü derecede her piksel için $2^3 = 8$ durum olacağını söyledi. Ayrıca 5 bitlik grayscale için "5 state", color table için "255 state" dedi.
  - Ders sonrası not: Yalnız bir önceki pikseli kullanan bu model, standart terminolojide büyük olasılıkla birinci dereceden Markov modeldir; k. derecede ikili görüntü için 2^k bağlam olur. 5 bitlik grayscale'de $2^5 = 32$, 256 renkli color table'da 256 durum vardır.
- **MTF kod çözme.** Slayttaki tabloda (ekrandan, 02:24:00) alfabenin son hâli `mnopabcd` ile başlanır ve kodlar sırayla okunur: `0 → m`, `1 → n`, `2 → o`, `3 → p`, `0 → p`, `1 → o`, `2 → n`, `3 → m`, `4 → a`, … Her çözülen sembol öne taşınır. Sonuç `m n o p p o n m a b c d d c b a`'dır, yani girdinin tersi; en sonda alfabe yeniden `abcdmnop` olur. Hoca bunu algoritmanın simetrik olmasıyla açıkladı. Slayttaki son satırda `abcdamnop` yazıyor; hoca fazladan `a`'yı sildi.
  - Ders sonrası not: Python ile denendi: başlangıç alfabesi `abcdmnop` ile kodlar baştan okunursa girdi doğrudan elde edilir. Slayttaki "son alfabeden başla" yolu bu örnekte doğru sonuç veriyor, çünkü girdinin tersi `mnopabcd` alfabesiyle kodlandığında da aynı kod dizisi çıkıyor. Standart MTF çözmede iki taraf büyük olasılıkla başlangıç alfabesini bilir ve kodlar baştan okunur.
  - Ders sonrası not: Hoca 02:22:06'da artıklığı "3 - 2,81 = 0,81" diye söyledi; doğrusu $3 - 2{,}81 = 0{,}19$ bittir.
- **Kelime MTF'de boşluklar.** Hoca kelime örneğinde boşlukların kodlanmadığını, istenirse boşluğun da kodlanabileceğini ya da açarken her kelimeden sonra bir boşluk bırakılabileceğini söyledi. Açmada çıktıda kelime varsa listeye eklenir, yoksa yalnız indeksteki kelime alınır; her iki durumda da kelime öne taşınır.

  - Ders sonrası not: Hoca 02:32:21'de son `boy` için "alfabenin sonuna getiriyorum" dedi; kodlamada kelime öne taşınır (slayttaki son satır `(boy, right, the, is, my, on)`).
- **Ön işlem örnekleri.** Hoca MTF'yi ön işlem olarak tanıtırken başka ön işlem örnekleri verdi: seste kulağın duymadığı frekansları kaldırmak, görüntüde parazitleri temizlemek, metinde çok tekrar eden boşlukları teke indirmek, kayıplı sıkıştırmada bütün karakterleri küçük harfe çevirmek. İstatistiksel yöntemler başlığı altında Huffman, aritmetik kodlama ve PPM'nin ileride görüleceğini söyledi.
  - Ders sonrası not: Hoca 02:12:33'te "fakslarda MNP5 ve 7 algoritmaları" dedi; MNP5 ve MNP7 büyük olasılıkla modem protokolleridir (Microcom Network Protocol).
- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.